

Differential fault analysis of ARIA in multi-byte fault models

Chong Hee Kim

Information Security Group, ICTEAM Institute, Université catholique de Louvain, Place Sainte Barbe, 2, Louvain-la-Neuve, Belgium

ARTICLE INFO

Article history:

Received 21 December 2011
Received in revised form 15 March 2012
Accepted 7 April 2012
Available online 20 April 2012

Keywords:

Cryptanalysis
Security
Differential fault analysis
Block cipher
ARIA

ABSTRACT

Differential fault analysis exploits faults to find secret information stored in a cryptographic device. It utilizes differential information between correct and faulty ciphertexts. We introduce new techniques to improve the previous differential fault analysis of ARIA. ARIA is a general-purpose involutory SPN (substitution permutation network) block cipher and was established as a Korean standard block cipher algorithm in 2004. While the previous method by Li et al. requires 45 faults, our method needs 13 faults to retrieve the 128-bit secret key of ARIA. If access to the decryption oracle is allowed, our method only needs 7 faults. We analyze the characteristics of the diffusion layer of ARIA in detail, which leads us to reduce the number of required faults to find the key.

© 2012 Elsevier Inc. All rights reserved.

1. Introduction

Reliable computation is one of the main concerns in many devices. Especially faults occurred during the operations cause many problems such as performance deterioration, unreliable output, etc. Hence, a lot of works to minimize, detect, or prevent faults have been researched. Nowadays, we can easily find cryptographic devices such as smart cards everywhere in our daily lives from banking cards to SIM cards for GSM. These devices are believed to be tamper-resistant. However, if a fault occurs, an adversary may find the secret information stored in the device. Therefore, we are challenging a new type of fault problem.

More precisely, an adversary can find the key of a block cipher using differential information between correct and faulty ciphertexts. This kind of attack is called *differential fault analysis* (DFA). A block cipher is widely used in many cryptographic applications and has been studied extensively in the literature. *Traditional cryptanalysis* of block cipher targets a cipher's design and architecture based on abstract and mathematical approaches. However, in practice a cipher has to be implemented on a real device that is exposed to *physical cryptanalysis* such as side-channel attacks (Dhem et al., 1998; Kocher et al., 1999; Quisquater and Samyde, 2001) and fault attacks (Bar-El et al., 2004; Kim and Quisquater, 2007).

An adversary gets faulty ciphertexts by giving external impact on a device with voltage variation, glitch, laser, etc. (Bar-El et al., 2004). The first DFA presented by Biham and Shamir (1997) targeted DES (National Institute of Standard and Technology, 1993).

The ways of exploiting faults to find the key are different according to each algorithm. Therefore, finding an efficient attack for each algorithm is main stream in the research of DFA. Up to now almost all cryptosystems, for example, Triple-DES (Hemme, 2004), RC4 (Biham et al., 2005; Hoch and Shamir, 2004), CLEFIA (Chen et al., 2007; Takahashi and Fukunaga, 2008), RSA (Coron et al., 2010), ElGamal (Bao et al., 1998), IDEA (Clavier et al., 2008), LUC and Demytko (Bleichenbacher et al., 1997), ECC (Blömer et al., 2005; Ciet and Joye, 2005), AES (Piret and Quisquater, 2003; Moradi et al., 2006; Kim and Quisquater, 2008; Takahashi et al., 2007; Barengi et al., 2010; Kim, 2010), SMS4 and MacGuffin (Li et al., 2009), DSA (Naccache et al., 2005), and ECDSA (Schmidt and Medwed, 2009; Barengi et al., 2011) have been broken.

The research of DFA can be further diversified into several directions: reducing the number of required faults, applying it to multi-byte fault models, extending to variants, if they exist, or exploring faults induced at an earlier round. In this article, we introduce new fault attacks on ARIA based on multi-byte fault models that needs less faults than the previous one.

ARIA is a general-purpose involutory SPN (substitution permutation network) block cipher algorithm, optimized for lightweight environments and hardware implementation. The name ARIA was taken from the initials of Academia, Research Institute and Agency, acknowledging the co-operative efforts of Korean researchers in designing ARIA. In 2004, ARIA was established as a Korean standard block cipher algorithm (KS X 1213) by the Ministry of Knowledge Economy (ARIA).

S_0^i	S_1^i	S_2^i	S_3^i
S_4^i	S_5^i	S_6^i	S_7^i
S_8^i	S_9^i	S_{10}^i	S_{11}^i
S_{12}^i	S_{13}^i	S_{14}^i	S_{15}^i

Fig. 1. State of ARIA.

Several traditional cryptanalysis (Wu et al., 2007; Li and Song, 2008; Li et al., 2008; Fleischmann et al., 2009) and side-channel analysis of ARIA (Ha et al., 2005; Kim et al., 2008; Park et al., 2007; Yoo et al., 2006) have been proposed. However, there is only one result of DFA of ARIA (Li et al., 2008). While the previous method by Li et al. (2008) requires 45 faults to retrieve the key, our method based on a two-byte fault model needs 13 faults. If access to the decryption oracle is allowed, our method needs 7 faults. Our generalized attack, working with faults corrupting a maximum of 4 bytes, can find the key with 21 faults.

This article is organized as follows: Section 2 introduces the ARIA algorithm. The next section briefly describes the previous work by Li et al. and explains our new techniques. Section 4 discusses possible countermeasures. Finally Section 5 concludes the article.

2. ARIA algorithm

ARIA is a 128-bit SPN block cipher¹ with 128-bit, 192-bit, or 256-bit key, where the number of rounds is 12, 14, and 16, respectively (ARIA, in press; Kwon et al., 2003). In the sequel, we will use the 128-bit key version of ARIA cipher, unless otherwise stated.

2.1. Structure of ARIA

The 128-bit input block passes through a round function, which is iterated 12 times (see Fig. 2). The intermediate cipher result, called *State*, can be represented as a two-dimensional byte array with 4 rows and 4 columns. The $S^i = (S_0^i, \dots, S_{15}^i)$ at round i is thus represented by an array as shown in Fig. 1.

The encryption and decryption processes are identical except the use of round keys. Each round consists of the following three parts:

- Round key addition: the State is XORed with a 128-bit subkey.
- Substitution layer (SL): the State goes through 16 S-boxes. There are 2 different substitutions, types 1 and 2, which alternate between the rounds.
- Diffusion layer (DL): it is a function which maps an input $(x_0, x_1, \dots, x_{15})$ of 16 bytes into an output $(y_0, y_1, \dots, y_{15})$. The mapping can also be considered as a 16×16 binary matrix multiplication as follows:

$$\begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \\ y_7 \\ y_8 \\ y_9 \\ y_{10} \\ y_{11} \\ y_{12} \\ y_{13} \\ y_{14} \\ y_{15} \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \\ x_9 \\ x_{10} \\ x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \\ x_{15} \end{pmatrix}$$

In the last round, instead of the diffusion layer, there is another key addition.

2.2. Key expansion of ARIA

The ARIA key expansion consists of two parts: *initialization* and *subkey generation*. In the initialization part, four 128-bit values, W_0 , W_1 , W_2 , and W_3 , are generated from the master key by using a 3-round Feistel cipher. Then the subkeys are generated by a sequence of XOR, rotate-right and rotate-left operations as follows:

$$\begin{aligned} ek_1 &= W_0 \oplus W_1^{\gg 19}, ek_2 = W_1 \oplus W_2^{\gg 19}, ek_3 = W_2 \oplus W_3^{\gg 19}, \\ ek_4 &= W_0^{\gg 19} \oplus W_3, ek_5 = W_0 \oplus W_1^{\gg 31}, ek_6 = W_1 \oplus W_2^{\gg 31}, \\ ek_7 &= W_2 \oplus W_3^{\gg 31}, ek_8 = W_0^{\gg 31} \oplus W_3, ek_9 = W_0 \oplus W_1^{\ll 61}, \\ ek_{10} &= W_1 \oplus W_2^{\ll 61}, ek_{11} = W_2 \oplus W_3^{\ll 61}, \\ ek_{12} &= W_0^{\ll 61} \oplus W_3, ek_{13} = W_0 \oplus W_1^{\ll 31}. \end{aligned}$$

The subkeys for decryption are derived from the subkeys for encryption as follows:

$$\begin{aligned} dk_1 &= ek_{13}, dk_2 = DL(ek_{12}), \dots, \\ dk_{12} &= DL(ek_2), dk_{13} = ek_1. \end{aligned}$$

2.3. Notations

We use the following notations to describe ARIA. We denote by $X \in (\{0, 1\}^8)^{16}$ the plaintext and by $Y \in (\{0, 1\}^8)^{16}$ the ciphertext. The i th subkey is denoted by $ek_i \in (\{0, 1\}^8)^{16}$, $1 \leq i \leq 13$. We denote by $A_i = (a_{0,i}, a_{1,i}, a_{2,i}, \dots, a_{15,i})$ and $B_i = (b_{0,i}, b_{1,i}, b_{2,i}, \dots, b_{15,i})$ the input and the output of the substitution layer at round i , $1 \leq i \leq 12$, respectively. We denote by $C_i = (c_{0,i}, c_{1,i}, c_{2,i}, \dots, c_{15,i})$ the output of the linear layer at round i , $1 \leq i \leq 12$ (see Fig. 2). We denote by $A_i^* = (a_{0,i}^*, a_{1,i}^*, a_{2,i}^*, \dots, a_{15,i}^*)$ and $B_i^* = (b_{0,i}^*, b_{1,i}^*, b_{2,i}^*, \dots, b_{15,i}^*)$ the faulty input and output of the substitution layer at round i , $1 \leq i \leq 12$, respectively. The faulty output of the linear layer at round i is denoted by $C_i^* = (c_{0,i}^*, c_{1,i}^*, c_{2,i}^*, \dots, c_{15,i}^*)$, $1 \leq i \leq 12$.

Let $\Delta A_i = (\Delta a_{0,i}, \Delta a_{1,i}, \Delta a_{2,i}, \dots, \Delta a_{15,i})$ be the difference between A_i and A_i^* , $1 \leq i \leq 12$. We denote by ΔB_i and ΔC_i the difference between B_i and B_i^* , and C_i and C_i^* , $1 \leq i \leq 12$, respectively. We denote by $SL(A_i)$ the output of 128-bit substitution layer for the input A_i , $1 \leq i \leq 12$. We denote by $SL^{-1}(B_i)$ the output of the inversion of 128-bit substitution layer for the input B_i , $1 \leq i \leq 12$. Let $DL(B_i)$ be the output of 128-bit diffusion layer for the input

¹ ARIA has three versions according to slightly different key expansion methods: V0.8, V0.9 and V1.0. We use the latest standard version, V1.0 (ARIA).

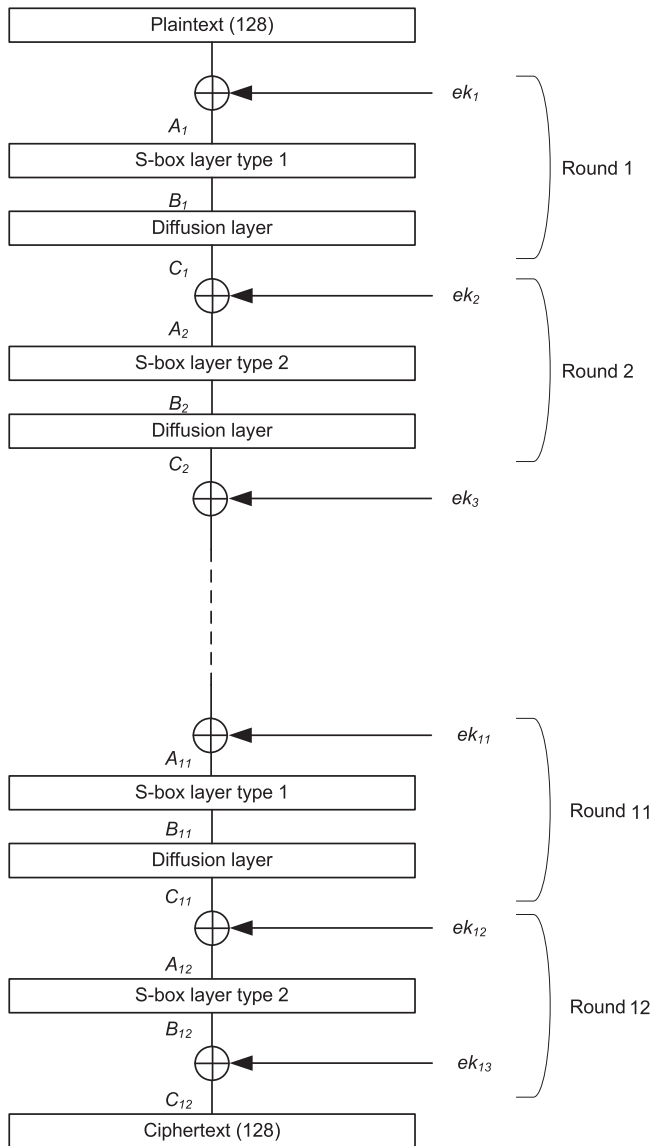


Fig. 2. Encryption process of ARIA.

B_i , $1 \leq i \leq 12$. We denote by $DL^{-1}(C_i)$ the output of the inversion of 128-bit diffusion layer for the input C_i , $1 \leq i \leq 12$.

3. New differential fault analysis on ARIA

3.1. Previous work

The ARIA diffusion layer propagates one single-byte fault to seven different bytes. That is, a single-byte fault induced before the diffusion layer at round 11 affects seven bytes of the ciphertext. Hence, one fault gives information on seven bytes of a subkey. Li et al. used this property and retrieved one subkey with 11 faulty ciphertexts on average and the master key with 45 faulty ciphertexts on average (Li et al., 2008). They directly applied “the method used to attack AES” in Piret and Quisquater (2003) to ARIA but did not analyze the characteristics of the ARIA diffusion layer that is different from the AES diffusion layer.

3.2. Fault models

We assume that an attacker induces a fault making 2 bytes corrupted (later it is generalized to a maximum of 4 bytes).

The corrupted bytes do not need to be consecutive. It is also assumed that the attacker obtains correct and faulty ciphertexts but does not know which bytes are corrupted. Furthermore, the faulty value is assumed to be random and uniformly distributed.

Our fault model is the same as that of Li et al. (2008) except the number of corrupted bytes. Instead of a single byte we assume that multiple bytes are corrupted. The multi-byte fault model has been applied to attack AES (Kim and Quisquater, 2008; Takahashi et al., 2007) and CLEFIA (Takahashi and Fukunaga, 2008; Zhao et al., 2010) and was shown to be practical by experiments (Fukunaga and Takahashi, 2009; Saha et al., 2009). In an 8-bit architecture one-byte fault model is desirable. However, depending on implementation, i.e., 16-bit/32-bit architecture or software implementation, multi-byte fault model can be much useful.

As shown in Table 1, our model is similar to those used to attack AES and CLEFIA. While some models assume that the number of corrupted bytes is fixed to 3 or 4 (Kim and Quisquater, 2008; Takahashi et al., 2007; Takahashi and Fukunaga, 2008), the others assume that the number of corrupted bytes is a maximum of 8, 12, or 16 (Saha et al., 2009; Zhao et al., 2010).

In Section 3.9, we use a generalized fault model that exploits any fault corrupting one, two, three, or four bytes simultaneously. It is also assumed that the attacker does not know how many bytes are corrupted and where the corrupted bytes are positioned. Normally the number of corrupted bytes varies each time a fault is induced (Saha et al., 2009). Therefore, we have to generate more faulty ciphertexts than expected in a fault model where the number of corrupted bytes is fixed. However, in our generalized fault model, we can exploit any fault corrupting a maximum of four bytes.

3.3. Basic idea

An attacker induces a random fault between C_{10} and B_{11} , and gets a faulty ciphertext. She obtains several pairs of correct and faulty ciphertexts by repeating it. Then, she finds the subkey of the last round, ek_{13} , (we will describe it in detail in Section 3.4) and computes A_{12} from the ciphertext and ek_{13} . She induces faults between C_9 and B_{10} and gets faulty ciphertexts. And she computes the subkey, ek_{12} . She repeats the procedure to the next round until she has enough subkeys to find the master key. In ARIA, she needs at least four subkeys, ek_{10} , ek_{11} , ek_{12} , and ek_{13} , as a 3-round Feistel cipher is used in the key expansion.

To help the readers understand our attack easily, we assume that 2 corrupted bytes are consecutive. We denote the errors induced on 2 bytes before the diffusion layer by (α, β) . These errors propagate through the diffusion layer as shown in Table 2, where $\eta_{\alpha\beta}$ presents that it is affected by both α and β . For example, let $(b_{0,i}, b_{1,i})$ be two bytes at round i . Then $(c_{3,i}, c_{4,i}, c_{6,i}, c_{13,i}, c_{14,i})$ are affected by $b_{0,i}$, $(c_{2,i}, c_{5,i}, c_{7,i}, c_{12,i}, c_{15,i})$ are affected by $b_{1,i}$, and $(c_{8,i}, c_{9,i})$ are affected by both $b_{0,i}$ and $b_{1,i}$. The bytes are exploitable only if they are affected by the same error and the number of affected bytes is at least two. In the previous example, there are 12 exploitable bytes (5 bytes affected by α , 5 bytes by β , and 2 bytes by both α and β). Hence, we can get information on 12 bytes of the subkey. However, if $(b_{3,i}, b_{4,i})$ or $(b_{11,i}, b_{12,i})$ are corrupted, we can get information on 10 bytes of the subkey. On average information on 11.73 bytes is available. We note that information on 7 bytes is available in the one-byte fault model of Li et al.

Note. We note again that our attack still works even when two corrupted bytes are *not* consecutive. This assumption is just for easy explanation. If they are not, we can find the subkeys by

Table 1
Multi-byte fault models.

Ref.	Target	Location	# of corrupted bytes
Kim and Quisquater (2008)	AES	Round $n - 1$ of key expansion	3
Takahashi et al. (2007)	AES	Round $n - 1$ of key expansion	4
Saha et al. (2009)	AES	Between rounds $n - 3$ and $n - 2$	1–8, 12, or 16
Takahashi and Fukunaga (2008)	CLEFIA	Round $n - 2$	4
Zhao et al. (2010)	CLEFIA	Rounds $n - 2$, $n - 1$, and n	1–8
This article	ARIA	Rounds $n - 4$, $\dots$, $n - 1$	1–4

constructing another propagation tables according to the position of the corrupted bytes.

3.4. Finding the subkey of the last round

An attacker induces a random fault between C_{10} and B_{11} making two bytes at B_{11} corrupted (see Fig. 2). Then she constructs three sets, $\mathbf{T}$, $\mathbf{U}$ and $\mathbf{V}$, of the positions of the bytes in A_{12} affected by the same error. We denote the size of each set by $m_{\mathbf{T}}$, $m_{\mathbf{U}}$, and $m_{\mathbf{V}}$, respectively. For example, if $(b_{0,11}, b_{1,11})$ are corrupted, $\mathbf{T} = \{3, 4, 6, 13, 14\}$, $\mathbf{U} = \{2, 5, 7, 12, 15\}$ and $\mathbf{V} = \{8, 9\}$ (see the first row of Table 2).

3.4.1. Finding $m_{\mathbf{T}} + m_{\mathbf{U}} + m_{\mathbf{V}}$ bytes of ek_{13}

An attacker finds $m_{\mathbf{T}} + m_{\mathbf{U}} + m_{\mathbf{V}}$ bytes of A_{12} . Then she finds $m_{\mathbf{T}} + m_{\mathbf{U}} + m_{\mathbf{V}}$ bytes of ek_{13} from the ciphertext and A_{12} . The input and output differences of the substitution layer of round 12 satisfy the following equation:

$$S(a_{j,12}) \oplus S(a_{j,12} \oplus \Delta a_{j,12}) = \Delta b_{j,12}, \quad (1)$$

where $0 \leq j \leq 15$. As the ciphertexts are known, $\Delta b_{j,12}$ can be computed.

For group $\mathbf{T}$, $(\Delta a_{t_1,12}, \dots, \Delta a_{t_{m_{\mathbf{T}}},12})$ can take only 255 values among $255^{m_{\mathbf{T}}}$ candidates as all $(a_{t_1,12}, \dots, a_{t_{m_{\mathbf{T}}},12})$ are affected by the same error. Eq. (1) can be rewritten as follows:

$$\Delta a_{j,12} = a_{j,12} \oplus S^{-1}(S(a_{j,12}) \oplus \Delta b_{j,12}). \quad (2)$$

By guessing $a_{j,12}$, she computes $\Delta a_{j,12}$ for $j = t_1, \dots, t_{m_{\mathbf{T}}}$ and checks whether $\Delta a_{t_1,12} = \Delta a_{t_2,12} = \dots = \Delta a_{t_{m_{\mathbf{T}}},12}$. If the equation is satisfied, she adds the guessed $a_{j,12}$ to the list $\mathcal{L}_{\mathbf{T}}$ of possible candidates. She repeats this for all possible values of $(a_{t_1,12}, \dots, a_{t_{m_{\mathbf{T}}},12})$. For $\mathbf{U}$ and $\mathbf{V}$, she repeats the procedure and gets $\mathcal{L}_{\mathbf{U}}$ and $\mathcal{L}_{\mathbf{V}}$, respectively. The complexity of computing $\mathcal{L}_{\mathbf{T}}$, $\mathcal{L}_{\mathbf{U}}$,

or $\mathcal{L}_{\mathbf{V}}$ is about $(2^8)^m$, $m=2, 3, 4$, or 5. Hence, the maximum complexity is 2^{40} , which it is impractical. However, we can reduce the complexity by slightly modifying the attack as shown in Algorithm 1.

Algorithm 1. Find $\mathcal{L}_{\mathbf{T}}$

Input: $\mathbf{T}$, $\Delta b_{j,12}$

Output: $\mathcal{L}_{\mathbf{T}}$

- Step 1. For each of the 2^{16} candidates of $(a_{t_1,12}, a_{t_2,12})$:
Compute $(\Delta a_{t_1,12}, \Delta a_{t_2,12})$ using Eq. (2).
If $\Delta a_{t_1,12} = \Delta a_{t_2,12}$, add it to the list $\mathcal{L}_{\mathbf{T}}$.
- Step 2. For each $l \in \mathcal{L}_{\mathbf{T}}$, try to extend it by one byte:
Remove l from $\mathcal{L}_{\mathbf{T}}$.
For all 2^8 of $a_{t_3,12}$:
Compute $\Delta a_{t_3,12}$.
If $\Delta a_{t_1,12} = \Delta a_{t_3,12}$, add the newly extended $(l, a_{t_3,12})$ to the list $\mathcal{L}_{\mathbf{T}}$.
- Step 3. Repeat Step 2 until elements of $\mathcal{L}_{\mathbf{T}}$ have a length of $m_{\mathbf{T}}$ bytes.

The number of candidates in $\mathcal{L}_{\mathbf{T}}$ after N pairs of ciphertexts have been treated is about $\lceil 256^{m_{\mathbf{T}}} / (255^{m_{\mathbf{T}}-1})^N \rceil$. With one pair of correct and faulty ciphertexts on average 260 candidates remain. With two pairs, we have one candidate on average. The number of candidates in $\mathcal{L}_{\mathbf{T}}$ in each step of Algorithm 1 is about 2^8 . The complexity of Algorithm 1 is $(2^{16}) \times 2(m_{\mathbf{T}} - 1)$ and hence, 2^{19} when $m_{\mathbf{T}}=5$. Therefore, it is practical to implement.

3.4.2. Finding all bytes of ek_{13}

With a pair of correct and faulty ciphertexts we can find 260 candidates for 11.73 bytes of A_{12} on average. If we have more faulty ciphertexts, we can further reduce the number of candidates. Furthermore, we can find more bytes of A_{12} by changing the position of the faulty bytes. Once A_{12} is found, the subkey is computed: $ek_{13} = SL(A_{12}) \oplus C_{12}$. Algorithm 2 shows the procedure in detail.

Table 2
Propagation of errors through the diffusion layer when two consecutive bytes are corrupted.

Errors before diff. layer	Propagated errors after diff. layer	#
$\alpha \beta 0000000000000000$	$00 \beta \alpha \alpha \beta \alpha \beta \eta_{\alpha\beta} \eta_{\alpha\beta} 00 \beta \alpha \alpha \beta$	12
$0 \alpha \beta 0000000000000000$	$0 \beta \alpha 0 \beta \alpha \beta \alpha \alpha \alpha \beta \beta \eta_{\alpha\beta} 00 \eta_{\alpha\beta}$	12
$00 \alpha \beta 0000000000000000$	$\beta \alpha 00 \alpha \beta \alpha \beta 00 \eta_{\alpha\beta} \eta_{\alpha\beta} \alpha \beta \beta \alpha$	12
$000 \alpha \beta 0000000000000000$	$\eta_{\alpha\beta} 0 \beta 00 \eta_{\alpha\beta} 0 \alpha \beta 0 \alpha \eta_{\alpha\beta} 0 \alpha \eta_{\alpha\beta} \beta$	10
$0000 \alpha \beta 0000000000000000$	$\alpha \beta \alpha \beta \beta \alpha 00 \alpha \beta \beta \alpha 00 \eta_{\alpha\beta} \eta_{\alpha\beta}$	12
$00000 \alpha \beta 0000000000000000$	$\beta \alpha \beta \alpha \alpha 00 \beta 0 \eta_{\alpha\beta} \eta_{\alpha\beta} 0 \beta \beta \alpha \alpha$	12
$000000 \alpha \beta 0000000000000000$	$\alpha \beta \alpha \beta 00 \beta \alpha \beta \alpha \alpha \beta \eta_{\alpha\beta} \eta_{\alpha\beta} 00$	12
$0000000 \alpha \beta 0000000000000000$	$\beta \eta_{\alpha\beta} 0 \alpha \beta 0 \alpha \beta \alpha 0 \beta \alpha \alpha \eta_{\alpha\beta} 0 \beta$	12
$00000000 \alpha \beta 0000000000000000$	$\eta_{\alpha\beta} \eta_{\alpha\beta} 00 \alpha \beta \beta \alpha 00 \alpha \beta \beta \alpha \beta \alpha$	12
$000000000 \alpha \beta 0000000000000000$	$\alpha \alpha \beta \beta 0 \eta_{\alpha\beta} \eta_{\alpha\beta} 0 \beta 00 \alpha \alpha \beta \alpha \beta$	12
$0000000000 \alpha \beta 0000000000000000$	$00 \eta_{\alpha\beta} \eta_{\alpha\beta} \beta \alpha \alpha \beta \alpha \beta 00 \beta \alpha \beta \alpha$	12
$00000000000 \alpha \beta 0000000000000000$	$0 \beta \eta_{\alpha\beta} \alpha \alpha 0 \beta \eta_{\alpha\beta} 0 \eta_{\alpha\beta} 0 \beta \eta_{\alpha\beta} 0 \alpha 0$	10
$000000000000 \alpha \beta 0000000000000000$	$\beta \alpha \alpha \beta 00 \eta_{\alpha\beta} \eta_{\alpha\beta} \beta \alpha \beta \alpha \alpha \beta 00$	12
$0000000000000 \alpha \beta 0000000000000000$	$\eta_{\alpha\beta} 00 \eta_{\alpha\beta} \beta \beta \alpha \alpha \alpha \beta \alpha \beta 0 \alpha \beta 0$	12
$00000000000000 \alpha \beta 0000000000000000$	$\alpha \beta \beta \alpha \eta_{\alpha\beta} \eta_{\alpha\beta} 00 \beta \alpha \beta \alpha 00 \alpha \beta$	12

Algorithm 2. Find ek_{13} Output: ek_{13}

Step 0. Let $k = 1$.
 Step 1. Induce a random fault between C_{10} and B_{11} making two bytes at B_{11} corrupted.
 Step 2. Find T_k , U_k , and V_k .
 Step 3. Compute the output difference, ΔB_{12} .
 Step 4. Find $\mathcal{L}_{T_k}$, $\mathcal{L}_{U_k}$, and $\mathcal{L}_{V_k}$ using Algorithm 1.
 Step 5. Reduce the number of candidates in $\mathcal{L}_{T_k}$, $\mathcal{L}_{U_k}$ and $\mathcal{L}_{V_k}$:
 If $k > 1$:
 For each $t_i^k \in T_k$:
 For each $t_j^n \in T_n$ with $1 \leq n \leq k-1$:
 Check whether $t_i^k = t_j^n$.
 If yes, compare the candidates in $\mathcal{L}_{T_k}$ and $\mathcal{L}_{T_n}$ for $a_{i,12}^k$.
 If the same candidate does not exist in both $\mathcal{L}_{T_k}$ and $\mathcal{L}_{T_n}$, remove it from the lists.
 Repeat for U_n and V_n .
 Repeat for U_k and V_k .
 Step 6. Construct a set $W = T_i \cup U_i \cup V_i$ for $1 \leq i \leq k$ and a list $\mathcal{L}_W$.
 Step 7. Increase k by one.
 Step 8. Repeat Step 1 to 7 until W has 16 elements and the number of candidates in $\mathcal{L}_W$ is 1.
 Step 9. Compute ek_{13} :
 $ek_{13} = SL(A_{12}) \oplus C_{12}$.

In Step 2, we can find sets of the positions of the bytes in A_{12} affected by the same error using Table 2. We know ΔC_{12} from a pair of correct and faulty ciphertexts and hence ΔB_{12} . However, we do not know ΔA_{12} ($= \Delta C_{11}$). If $\Delta c_{j,11}$ is equal to 0, the corresponding $\Delta a_{j,12} = \Delta b_{j,12} = \Delta c_{j,12} = 0$, $0 \leq j \leq 15$, and vice versa. We call the pattern of zero bytes at ΔC_{12} *zero pattern*. Then with *zero pattern* in ΔC_{12} , we can find the position of the corrupted bytes. For example, if $(b_{0,11}, b_{1,11})$ are corrupted at B_{11} , we have a *zero pattern* of (0 0 - - - - - 0 0 - - - -) at ΔC_{12} .

Four cases have the same *zero patterns*: (b_0, b_1) and (b_{10}, b_{11}) , (b_2, b_3) and (b_8, b_9) , (b_4, b_5) and (b_{14}, b_{15}) , and (b_6, b_7) and (b_{12}, b_{13}) . For example, if $\Delta C_{12} = (0 0 - - - - - 0 0 - - - -)$, the corrupted bytes are either (b_0, b_1) or (b_{10}, b_{11}) . If an attacker induces a fault during the first half of the operations in the layer, it is (b_0, b_1) . If she does during the second half, it is (b_{10}, b_{11}) . Hence, she can find the location of a fault. So can she for the other 3 cases.

Step 5 discards impossible candidates. We explain how Step 5 works with an example. We assume that the first fault corrupts $(b_{0,11}, b_{1,11})$. Then, $T_1 = \{3, 4, 6, 13, 14\}$, $U_1 = \{2, 5, 7, 12, 15\}$ and $V_1 = \{8, 9\}$. After Step 4, $\mathcal{L}_{T_1}$, $\mathcal{L}_{U_1}$, and $\mathcal{L}_{V_1}$ have about 260 candidates respectively. We assume that the second fault corrupts $(b_{4,11}, b_{5,11})$. Then $T_2 = \{0, 2, 5, 8, 11\}$, $U_2 = \{1, 3, 4, 9, 10\}$, and $V_2 = \{14, 15\}$. Step 5 first finds elements included in both T_1 and T_2 . As $T_1 \cap T_2 = \emptyset$, it compares T_1 and U_2 and $T_1 \cap U_2 = \{3, 4\}$. Then, it compares the candidates in $\mathcal{L}_{T_1}$ and those in $\mathcal{L}_{U_2}$ for $a_{3,12}$ and $a_{4,12}$. If a candidate does not belong to both $\mathcal{L}_{T_1}$ and $\mathcal{L}_{U_2}$ simultaneously, it is removed from the lists. It continues the process and $T_1 \cap V_2 = \{14\}$, $U_1 \cap T_2 = \{2, 5\}$, $U_1 \cap U_2 = \emptyset$, $U_1 \cap V_2 = \{15\}$, $V_1 \cap T_2 = \{8\}$, $V_1 \cap U_2 = \{9\}$, and $V_1 \cap V_2 = \emptyset$. As two pairs of correct and faulty ciphertexts allow retrieving a unique value, we can find a unique candidate for $a_{3,12}$ and $a_{4,12}$. We also have one candidate for $T_1 \cup U_2$. Hence, we can find $(a_{1,12}, a_{3,12}, a_{4,12}, a_{6,12}, a_{9,12}, a_{10,12}, a_{13,12}, a_{14,12})$. Similarly, we have one candidate for $(a_{0,12}, a_{2,12}, a_{5,12}, a_{7,12}, a_{8,12}, a_{11,12}, a_{12,12}, a_{15,12})$ from U_1 and U_2 . Finally, we have $W = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14, 15\}$ and one candidate in $\mathcal{L}_W$. Therefore, Algorithm 2 outputs a single candidate for ek_{13} .

3.5. Finding the master key

We describe how to find the ARIA master key using the method introduced in Section 3.4. Once we have found ek_{13} , we can compute A_{12} and remove the last round. Then we apply Algorithm 2

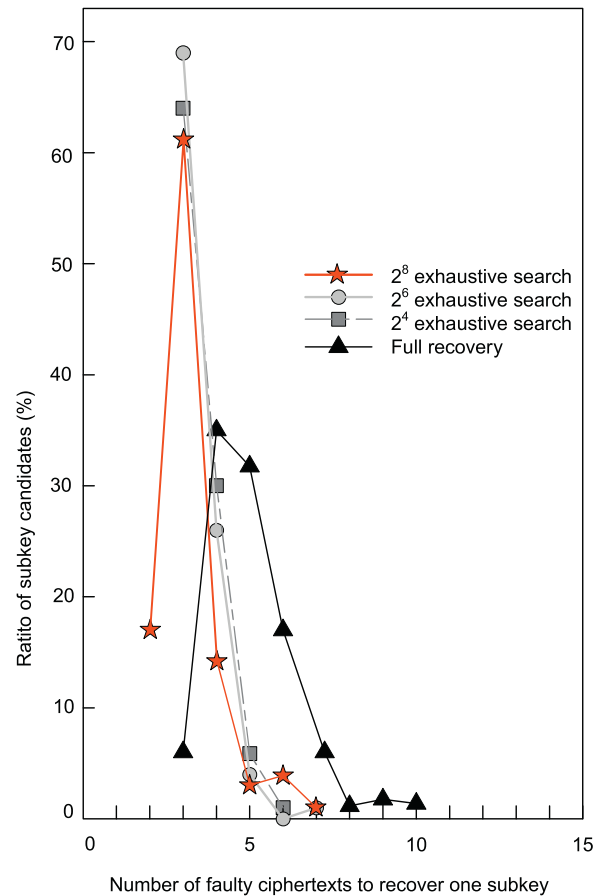


Fig. 3. Ratio of faulty ciphertexts to retrieve 1 subkey.

to find ek_{12} . We repeat this until we have found 4 subkeys. In summary, we have the following steps:

- Step 1. Find ek_{13} :
 - Induce a random error between C_{10} and B_{11} and get a pair of correct and faulty ciphertexts.
 - Get several pairs of correct and faulty ciphertexts by repeating it.
 - Find A_{12} .
 - Compute $ek_{13} = SL(A_{12}) \oplus C_{12}$, where $C_{12} = Y$.
 Step 3. Find ek_{12} :
 - Induce a random error between C_9 and B_{10} .
 - Get several pairs of correct and faulty ciphertexts by repeating it.
 - Find A_{11} .
 - Compute $ek_{12} = DL(SL(A_{11})) \oplus A_{12}$.
 Step 4. Similarly find ek_{11} and ek_{10} .
 Step 5. Find the master key using the key expansion with ek_{10} , ek_{11} , ek_{12} , and ek_{13} .

3.6. Simulation results

We implemented our attack on a PC with a 3.20GHz Intel and 8GB memory using Visual C++7.1 Compiler. The fault was simulated by a computer software. We ran the algorithm to 100 encryption units with randomly generated keys.

Referring to Fig. 3, the number of required faulty ciphertexts to retrieve 1 subkey changes between 3 and 10 (see “Full recovery” in Fig. 3). Hence, we can retrieve a subkey with 3 faulty ciphertexts in the best case and with 10 faulty ciphertexts in the worst case, respectively. On average 4.98 ciphertexts are required. To recover the master key, we need 20 faulty ciphertexts on average. The complexity is about $2^{24} \approx 2^{19} \times 20$. The time to complete the attack was a few seconds.

Table 3

Propagation of errors through the diffusion layer when three consecutive bytes are corrupted.

Errors before diff. layer	Propagated errors after diff. layer	#
$\alpha \beta \gamma 00000000000000$	$0 \gamma \beta \alpha \eta_{\alpha\gamma} \beta \eta_{\alpha\gamma} \beta \eta_{\alpha\beta} \eta_{\alpha\beta} \gamma \gamma \eta_{\beta\gamma} \alpha \alpha \eta_{\beta\gamma}$	15
$0 \alpha \beta \gamma 00000000000000$	$\gamma \beta \alpha 0 \beta \eta_{\alpha\gamma} \beta \eta_{\alpha\gamma} \alpha \alpha \eta_{\beta\gamma} \eta_{\beta\gamma} \eta_{\alpha\beta} \gamma \gamma \eta_{\alpha\beta}$	15
$00 \alpha \beta \gamma 00000000000000$	$\eta_{\beta\gamma} \alpha \gamma 0 \alpha \eta_{\beta\gamma} \alpha \beta \gamma 0 \eta_{\alpha\beta} \eta_{\alpha\beta\gamma} \alpha \beta \eta_{\beta\gamma} \eta_{\alpha\gamma}$	11
$000 \alpha \beta \gamma 00000000000000$	$\eta_{\alpha\beta} \gamma \beta \gamma \gamma \eta_{\alpha\beta} 0 \alpha \beta \gamma \eta_{\alpha\gamma} \eta_{\alpha\beta} 0 \alpha \eta_{\alpha\beta\gamma} \eta_{\beta\gamma}$	11
$0000 \alpha \beta \gamma 00000000000000$	$\eta_{\alpha\gamma} \beta \eta_{\alpha\gamma} \beta \beta \alpha 0 \gamma \alpha \eta_{\beta\gamma} \eta_{\beta\gamma} \alpha \gamma \gamma \eta_{\alpha\beta} \eta_{\alpha\beta}$	15
$00000 \alpha \beta \gamma 00000000000000$	$\beta \eta_{\alpha\gamma} \beta \eta_{\alpha\gamma} \alpha 0 \gamma \beta \gamma \eta_{\alpha\beta} \eta_{\alpha\beta} \gamma \eta_{\beta\gamma} \eta_{\beta\gamma} \alpha \alpha$	15
$000000 \alpha \beta \gamma 00000000000000$	$\eta_{\alpha\gamma} \eta_{\beta\gamma} \alpha \beta \gamma 0 \beta \eta_{\alpha\gamma} \beta \alpha \eta_{\alpha\gamma} \beta \eta_{\alpha\beta} \eta_{\alpha\beta\gamma} 0 \gamma$	11
$0000000 \alpha \beta \gamma 00000000000000$	$\eta_{\beta\gamma} \eta_{\alpha\beta\gamma} 0 \alpha \beta \gamma \eta_{\alpha\gamma} \beta \alpha 0 \beta \eta_{\alpha\gamma} \eta_{\alpha\gamma} \eta_{\alpha\beta} \gamma \beta$	11
$00000000 \alpha \beta \gamma 00000000000000$	$\eta_{\alpha\beta} \eta_{\alpha\beta} \gamma \gamma \alpha \eta_{\beta\gamma} \eta_{\beta\gamma} \alpha \gamma 0 \alpha \beta \beta \eta_{\alpha\gamma} \beta \eta_{\alpha\gamma}$	15
$000000000 \alpha \beta \gamma 00000000000000$	$\alpha \alpha \eta_{\beta\gamma} \eta_{\beta\gamma} \gamma \eta_{\alpha\beta} \eta_{\alpha\beta} \gamma \beta \gamma 0 \alpha \eta_{\alpha\gamma} \beta \eta_{\alpha\gamma} \beta$	15
$0000000000 \alpha \beta \gamma 00000000000000$	$0 \gamma \eta_{\alpha\beta\gamma} \eta_{\alpha\beta} \beta \alpha \eta_{\alpha\gamma} \eta_{\beta\gamma} \alpha \eta_{\beta\gamma} 0 \gamma \eta_{\beta\gamma} \alpha \beta \alpha$	11
$00000000000 \alpha \beta \gamma 00000000000000$	$\gamma \beta \eta_{\alpha\beta} \eta_{\alpha\gamma} \alpha 0 \eta_{\beta\gamma} \eta_{\alpha\beta\gamma} \gamma \eta_{\alpha\beta} \gamma \beta \eta_{\alpha\beta} \gamma \alpha 0$	11
$000000000000 \alpha \beta \gamma 00000000000000$	$\eta_{\beta\gamma} \alpha \alpha \eta_{\beta\gamma} \gamma \gamma \eta_{\alpha\beta} \eta_{\alpha\beta} \beta \eta_{\alpha\gamma} \beta \eta_{\alpha\gamma} \alpha \beta \gamma 0$	15
$0000000000000 \alpha \beta \gamma 00000000000000$	$\eta_{\alpha\beta} \gamma \gamma \eta_{\alpha\beta} \eta_{\beta\gamma} \eta_{\beta\gamma} \alpha \alpha \eta_{\alpha\gamma} \beta \eta_{\alpha\gamma} \beta 0 \alpha \beta \gamma$	15

3.7. Further reducing the number of faults

We need 20 faulty ciphertexts on average to find the ARIA 128-bit key. In this section, we describe methods to further reduce the number of required faulty ciphertexts.

3.7.1. Partial search and brute-force attack

We may stop searching candidates of the subkey before the number of candidates is equal to one, and find the correct key by exhaustively searching the remaining candidates. This is done by changing “the number of candidates in $\mathcal{L}_W$ is 1” for “the number of candidates in $\mathcal{L}_W \leq 2^4, 2^6$, or 2^8 ” in Step 8 of Algorithm 2. Then Algorithm 2 outputs $2^4, 2^6$, or 2^8 candidates instead of 1. The number of required faulty ciphertexts for each case is shown in Fig. 3. We can see that the number of faulty ciphertexts reduces as the number of exhaustive search increases. On average 3.45, 3.38, and 3.19 ciphertexts are required, respectively.

As 2^8 exhaustive searches are required to find 1 subkey, 2^{32} searches are required to find the master key. The search of 2^{32} candidates is trivial from a cryptanalytic point of view, which means that the key is broken. For example, it took 8–35 min with a Core2 Duo 3.0 GHz PC (Fukunaga and Takahashi, 2009). Therefore, we can retrieve the master key with on average 13 faulty ciphertexts and 2^{32} exhaustive searches.

3.7.2. Access to the decryption oracle

If access to the decryption oracle is allowed, the attack can be transposed into the initial rounds of the cipher (Rivain, 2009). In fact, an attacker may obtain a faulty ciphertext Y^* from a plaintext X by inducing a fault at the end of the first round. The plaintext X can then be viewed as the faulty result of a decryption of Y^* for which a fault has been induced at the beginning of the last round. She then asks for the decryption of Y^* which provides her with a

plaintext X^* . The pair (X, X^*) thus constitutes a pair of correct and faulty results of the decryption algorithm with respect to an error induced at the beginning of the last round. Therefore, we can find dk_{13} with 4 faults as described in the previous section. We can also find ek_{13} with additional 4 faults without accessing the decryption oracle.

Then from ARIA key expansion, we know that

$$ek_1 = dk_{13}, ek_1 = W_0 \oplus W_1^{\gg 19}, ek_{13} = W_0 \oplus W_1^{\ll 31}.$$

The 128-bit master key can be easily computed from ek_1 and ek_{13} (for the detailed explanation we refer to Ha et al. (2005)). Therefore, we need 7 faults to find the master key if an attacker can access to the decryption oracle.

3.8. Analysis when three or four bytes are corrupted

We know that we can get information on 7 bytes of a subkey with a one-byte fault and 11.73 bytes with a two-byte fault. We compute how many information we can get with a three-byte or a four-byte fault (see Tables 3 and 4, respectively.)

To be exploited, at least two bytes should be affected by the same error. Furthermore, the location of the fault should be traceable from the output difference. For example, if $(b_{2,i}, b_{3,i}, b_{4,i}, b_{5,i})$ are four corrupted bytes at round i (see the third row of Table 4), they are useless since there is no zero pattern and hence it is impossible to find the location of the corrupted bytes. Finally, the information we can get with a three-byte fault is 13.29 bytes on average. That with a four-byte fault is 3.23 bytes on average.

3.9. Generalized fault model

Until now we have assumed that two bytes are corrupted. We loosen this constraint and assume that a maximum of four bytes

Table 4

Propagation of errors through the diffusion layer when four consecutive bytes are corrupted.

Errors before diff. layer	Propagated errors after diff. layer	#
$\alpha \beta \gamma \delta 00000000000000$	$\delta \gamma \beta \alpha \eta_{\alpha\gamma} \eta_{\beta\delta} \eta_{\alpha\gamma} \eta_{\beta\delta} \eta_{\alpha\beta} \eta_{\alpha\beta} \eta_{\gamma\delta} \eta_{\gamma\delta} \eta_{\beta\gamma} \eta_{\alpha\delta} \eta_{\alpha\delta} \eta_{\beta\gamma}$	0
$0 \alpha \beta \gamma \delta 00000000000000$	$\eta_{\gamma\delta} \beta \eta_{\alpha\delta} 0 \beta \eta_{\alpha\gamma\delta} \beta \eta_{\alpha\gamma} \eta_{\alpha\delta} \alpha \eta_{\beta\gamma} \eta_{\beta\gamma\delta} \eta_{\alpha\beta} \gamma \eta_{\gamma\delta} \eta_{\alpha\beta\delta}$	7
$00 \alpha \beta \gamma \delta 00000000000000$	$\eta_{\beta\gamma} \eta_{\alpha\delta} \gamma \delta \eta_{\alpha\delta} \eta_{\beta\gamma} \alpha \beta \gamma \delta \eta_{\alpha\beta\delta} \eta_{\alpha\beta\gamma} \alpha \beta \eta_{\beta\gamma\delta} \eta_{\alpha\gamma\delta}$	0
$000 \alpha \beta \gamma \delta 00000000000000$	$\eta_{\alpha\beta\delta} \gamma \eta_{\beta\delta} \gamma \gamma \eta_{\alpha\beta} 0 \eta_{\alpha\delta} \beta \eta_{\gamma\delta} \eta_{\alpha\gamma\delta} \eta_{\alpha\beta} \delta \eta_{\alpha\delta} \eta_{\alpha\beta\gamma} \eta_{\beta\gamma}$	7
$0000 \alpha \beta \gamma \delta 00000000000000$	$\eta_{\alpha\gamma} \eta_{\beta\delta} \eta_{\alpha\gamma} \eta_{\beta\delta} \beta \alpha \delta \gamma \eta_{\alpha\delta} \eta_{\beta\gamma} \eta_{\beta\gamma} \eta_{\alpha\delta} \eta_{\gamma\delta} \eta_{\gamma\delta} \eta_{\alpha\beta} \eta_{\alpha\beta}$	0
$00000 \alpha \beta \gamma \delta 00000000000000$	$\eta_{\beta\delta} \eta_{\alpha\gamma\delta} \beta \eta_{\alpha\gamma} \eta_{\alpha\delta} 0 \gamma \eta_{\beta\delta} \gamma \eta_{\alpha\beta} \eta_{\alpha\beta\delta} \gamma \eta_{\beta\gamma} \eta_{\beta\gamma\delta} \alpha \eta_{\alpha\delta}$	7
$000000 \alpha \beta \gamma \delta 00000000000000$	$\eta_{\alpha\gamma\delta} \eta_{\beta\gamma\delta} \alpha \beta \gamma \delta \eta_{\beta\delta} \eta_{\alpha\gamma} \beta \alpha \eta_{\alpha\gamma} \eta_{\beta\delta} \eta_{\alpha\beta\delta} \eta_{\alpha\beta\gamma} \delta \gamma$	0
$0000000 \alpha \beta \gamma \delta 00000000000000$	$\eta_{\beta\gamma} \eta_{\alpha\beta\gamma} \delta \eta_{\alpha\delta} \beta \eta_{\gamma\delta} \eta_{\alpha\gamma\delta} \beta \eta_{\alpha\delta} 0 \beta \eta_{\alpha\gamma} \eta_{\alpha\gamma} \eta_{\alpha\beta\delta} \gamma \eta_{\beta\delta}$	7
$00000000 \alpha \beta \gamma \delta 00000000000000$	$\eta_{\alpha\beta} \eta_{\alpha\beta} \eta_{\gamma\delta} \eta_{\gamma\delta} \eta_{\alpha\delta} \eta_{\beta\gamma} \eta_{\beta\gamma} \eta_{\alpha\delta} \gamma \delta \alpha \beta \eta_{\beta\delta} \eta_{\alpha\gamma} \eta_{\beta\delta} \eta_{\alpha\gamma}$	0
$000000000 \alpha \beta \gamma \delta 00000000000000$	$\alpha \eta_{\alpha\delta} \eta_{\beta\gamma\delta} \eta_{\beta\gamma} \gamma \eta_{\alpha\beta} \eta_{\alpha\beta\delta} \eta_{\gamma\delta} \beta \eta_{\gamma\delta} 0 \eta_{\alpha\delta} \eta_{\alpha\gamma\delta} \beta \eta_{\alpha\gamma} \beta$	7
$0000000000 \alpha \beta \gamma \delta 00000000000000$	$\delta \gamma \eta_{\alpha\beta\gamma} \eta_{\alpha\beta\delta} \beta \alpha \eta_{\alpha\gamma\delta} \eta_{\beta\gamma\delta} \eta_{\alpha\delta} \eta_{\beta\gamma} \delta \gamma \eta_{\beta\gamma} \eta_{\alpha\delta} \beta \alpha$	0
$00000000000 \alpha \beta \gamma \delta 00000000000000$	$\eta_{\gamma\delta} \beta \eta_{\alpha\beta} \eta_{\alpha\gamma\delta} \eta_{\alpha\delta} \delta \eta_{\beta\gamma} \eta_{\alpha\beta\gamma} \gamma \eta_{\alpha\beta\delta} \gamma \eta_{\beta\delta} \eta_{\alpha\beta} \gamma \eta_{\alpha\delta} 0$	7
$000000000000 \alpha \beta \gamma \delta 00000000000000$	$\eta_{\beta\gamma} \eta_{\alpha\delta} \eta_{\alpha\delta} \eta_{\beta\gamma} \eta_{\gamma\delta} \eta_{\gamma\delta} \eta_{\alpha\beta} \eta_{\beta\delta} \eta_{\alpha\gamma} \eta_{\beta\delta} \eta_{\alpha\gamma} \alpha \beta \gamma \delta$	0

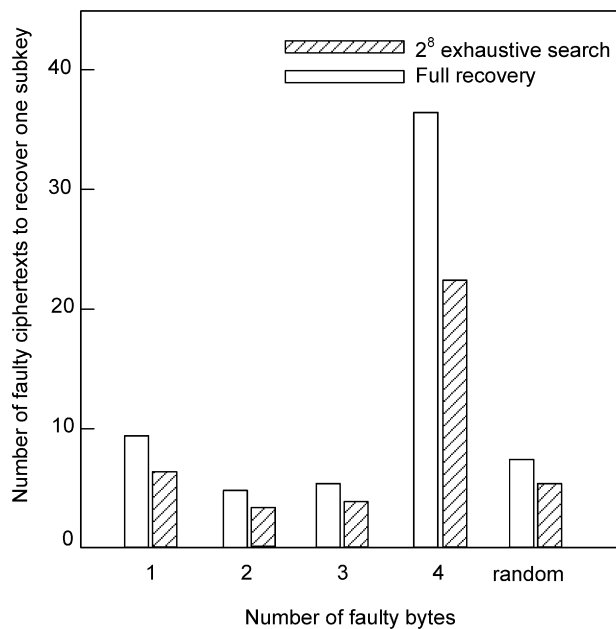


Fig. 4. Number of faulty ciphertexts to recover one subkey.

are corrupted. Furthermore, it is assumed that an attacker does not know the number of corrupted bytes.

Then we can find the subkeys and the master key by slightly modifying Algorithm 2. In Step 2, we first find the number and the location of corrupted bytes, and the sets of the positions of the bytes affected by the same error with the fault distribution tables (we can easily find them from the *zero pattern* at the output with Tables 1–3). Once we have found the sets, we follow the other steps but need to modify some parts according to the number of sets.

Fig. 4 and Table 5 show the simulation results. We first tried to find the number of faulty ciphertexts required to retrieve all 16 bytes of a subkey (we call it *full recovery*). Then we found the number of faulty ciphertexts required for a partial search remaining 2^8 exhaustive searches (hence, overall exhaustive search to find the master key becomes 2^{32}). The number of required faulty ciphertexts for full recovery when one byte is corrupted is 9.72. We need 6.38 faulty ciphertexts if 2^8 exhaustive searches are allowed. Our simulation results show a better performance than that of Li et al. (2008) in a one-byte fault model as the *comparison-and-reduction process* (Step 5 of Algorithm 2) removes impossible candidates.

Algorithm 2 eliminates wrong candidates quickly as the size of the sets of the positions of the bytes affected by the same error is bigger. Therefore, a three-byte fault is less efficient than a two-byte fault even if the number of useful bytes, 13.29, is bigger than that of the two-byte fault, 11.73. While 58 % of the sets have a size of 5 and 29 % have a size of 3 in a two-byte fault model, 36 % have a size of 3 and 55 % have a size of 2 in a three-byte fault model.

When an attacker does not know the number and the position of faulty bytes (we call it *random-byte* fault model), the number of required faulty ciphertexts to retrieve the master key is 5.26 on average allowing 2^8 exhaustive searches. Therefore, we need 21 faulty ciphertexts on average with 2^{32} exhaustive searches to find the 128-bit master key. The number of corrupted bytes varies

depending on the structure of the implementation, type of injected faults, etc. A fault should corrupt only single byte in the previous work by Li et al. (2008). However, our attack based on a *random-byte* fault model works if a fault corrupts a maximum of four bytes. Furthermore, it requires less faulty ciphertexts.

4. Possible countermeasures

Several countermeasures to detect faults have been proposed in the literature. Especially, three techniques – redundancy-based, code-based, and pattern-based techniques – are mentioned in Li et al. (2008). In this section, we examine whether these countermeasures can prevent our attacks in multi-byte fault models.

Redundancy-based technique is to perform additional encryption (or decryption), and then check whether the result is the same (or the original data). As all DFA exploit faults induced at the first or the last few rounds, this solution may compute the first and the last few rounds again instead of re-encrypting the full rounds. Code-based solutions are divided into coding method and error detection code (EDC). Coding method encodes message before encryption and checks errors after decryption. EDC approach is often used in each rounds' inner parts with the implementation of parity-based EDC. Li et al. proposed pattern-based technique that utilizes the ciphertext difference (Li et al., 2008). It checks pattern of zero and non-zero bytes in the ciphertext difference after assuming that one byte is corrupted.

Redundancy-based and code-based techniques are independent of the adopted fault model. Hence, these techniques are applicable to prevent our attacks. Li et al.'s pattern-based technique can also be used to prevent our attacks as we can construct pattern of zero and non-zero bytes in the ciphertext difference as shown in Tables 2–4. However, we note that pattern-based technique is not so much efficient because it should store many ciphertexts. An attacker may give a fault after several iterations of encryptions. Hence, in order to detect a fault it needs the XOR result of all possible combination of two ciphertexts during certain period.

5. Conclusions and discussion

We introduced new differential fault attacks on ARIA. By assuming that an attacker induces a fault corrupting two bytes, we need 13 faults on average to recover the ARIA 128-bit secret key. If access to the decryption oracle is allowed, we need 7 faults. The attacker does not need to know the position and value of the corrupted bytes in advance.

Furthermore, we proposed a generalized technique that works even if a fault corrupts a maximum of four bytes. The attacker can find the key with 21 faulty ciphertexts without knowing the number of corrupted bytes. This improvement comes from our analysis of the characteristics of the ARIA diffusion layer that has not been carried out before.

One interesting point we have found is that the enhancement in terms of security against traditional cryptanalysis may be helpful in differential fault analysis. ARIA has very similar structure to AES except the diffusion layer. AES diffusion layer diffuses one byte to four bytes. However, ARIA diffusion layer does one byte to seven bytes in order to amplify the diffusion effect, which leaks more information on the key in differential fault analysis. This is the case for faults induced at one round earlier.

The best known DFA on AES exploits faults induced at two rounds earlier, hence the affected bytes after two rounds become 16 bytes. In ARIA, the analysis of faults induced at two rounds earlier is more complicated due to complex key expansion as well as asymmetric structure of the diffusion layer. AES has a simple key scheduling method so that knowing the last round key allows to

Table 5
Number of faulty ciphertexts to find a subkey for each fault model.

# of faulty bytes	1	2	3	4	Random
Full recovery	9.72	4.98	5.61	36.82	7.53
2^8 exhaustive search	6.38	3.19	4.09	22.65	5.26

find the previous round key. Hence, the key space for the consecutive two rounds is 2^{128} . However, ARIA has 2^{256} key space for two consecutive round keys. Anyhow, the analysis of faults induced at two rounds earlier will be good research direction for future work.

Acknowledgements

The author thanks the reviewers for their helpful comments. This work has been partially funded by the Walloon Region Marshall Plan through the SPW DG06 Project TRASILUX.

References

- ARIA, Korean standard block cipher algorithm. Available from: <http://210.104.33.10/ARIA/index-e.html>.
- Bao, F., Deng, R.H., Han, Y., Jeng, A.B., Narasimhalu, A.D., Ngair, T.-H., 1998. Breaking public key cryptosystems on tamper resistant devices in the presence of transient faults. In: The 5th International Workshop on Security Protocols, vol. 1361 of Lecture Notes in Computer Science. Springer, pp. 115–124.
- Bar-El, H., Choukri, H., Naccache, D., Tunstall, M., Whelan, C., 2004. The sorcerer's apprentice guide to fault attacks. In: Fault Diagnosis and Tolerance in Cryptography in association with the International Conference on Dependable Systems and Networks (DSN 2004), pp. 330–342.
- Barengi, A., Bertoni, G., Breviglieri, L., Pellicoli, M., Pelosi, G., 2010. Low voltage fault attacks to AES. In: IEEE International Symposium on Hardware-Oriented Security and Trust – HOST 2010. IEEE Computer Society, pp. 7–12.
- Barengi, A., Bertoni, G., Palomba, A., Susella, R., 2011. A novel fault attack against ECDSA. In: IEEE International Symposium on Hardware-Oriented Security and Trust – HOST 2011. IEEE Computer Society, pp. 161–166.
- Biham, E., Granboulan, L., Nguyen, P.Q., 2005. Impossible fault analysis of RC4 and differential fault analysis of RC4. In: Fast Software Encryption: 12th International Workshop, FSE 2005, vol. 3557 of Lecture Notes in Computer Science. Springer, pp. 359–367.
- Biham, E., Shamir, A., 1997. Differential fault analysis of secret key cryptosystems. In: Advances in Cryptology – CRYPTO '97, 17th Annual International Cryptology Conference, vol. 1294 of Lecture Notes in Computer Science. Springer, pp. 513–525.
- Blömer, J., Otto, M., Seifert, J.-P., 2005. Sign change fault attacks on elliptic curve cryptosystems. In: The 2nd International Workshop on Fault Diagnosis and Tolerance in Cryptography – FDTCT 2005, pp. 25–40.
- Bleichenbacher, D., Joye, M., Quisquater, J.-J., 1997. A new and optimal chosen-message attack on RSA-type cryptosystems. In: Information and Communication Security, First International Conference, ICICS'97, vol. 1334 of Lecture Notes in Computer Science. Springer, pp. 302–313.
- Chen, H., Wu, W., Feng, D., 2007. Differential fault analysis on CLEFIA. In: Information and Communications Security, 9th International Conference, ICICS 2007, vol. 4861 of Lecture Notes in Computer Science. Springer, pp. 284–295.
- Ciet, M., Joye, M., 2005. Elliptic curve cryptosystems in the presence of permanent and transient faults. Designs, Codes and Cryptography 36 (1), 33–43.
- Clavier, C., Gierlichs, B., Verbauwhede, I., 2008. Fault analysis study of IDEA. In: Topics in Cryptology – CT-RSA 2008, The Cryptographer's Track at RSA Conference 2008, vol. 4964 of Lecture Notes in Computer Science. Springer, pp. 274–287.
- Coron, J.-S., Giraud, C., Morin, N., Piret, G., Vigilant, D., 2010. Fault attacks and countermeasures on Vigilant's RSA-CRT algorithm. In: Fault Diagnosis and Tolerance in Cryptography, 7th International Workshop, FDTCT 2010. IEEE Computer Society, pp. 89–96.
- Dhem, J.-F., Koene, F., Leroux, P.-A., Mestré, P., Quisquater, J.-J., Willems, J.-L., 1998. A practical implementation of the timing attack. In: Smart Card Research and Applications, This International Conference, CARDIS '98, vol. 1820 of Lecture Notes in Computer Science. Springer, pp. 167–182.
- Fleischmann, E., Gorski, M., Lucks, S., 2009. Attacking reduced rounds of the ARIA block cipher. IACR Eprint Archive 200, 9–334.
- Fukunaga, T., Takahashi, J., 2009. Practical fault attack on a cryptographic LSI with ISO/IEC 18033-3 block ciphers. In: 6th International Workshop on Fault Diagnosis and Tolerance in Cryptography, FDTCT 2009. IEEE Computer Society, pp. 84–92.
- Ha, J., Kim, C., Moon, S.-J., Park, I., Yoo, H., 2005. Differential power analysis on block cipher ARIA. In: HPCC, vol. 3726 of Lecture Notes in Computer Science. Springer, pp. 541–548.
- Hemme, L., 2004. A differential fault attack against early rounds of (triple-)DES. In: Cryptographic Hardware and Embedded Systems – CHES 2004: 6th International Workshop, vol. 3156 of Lecture Notes in Computer Science. Springer, pp. 254–267.
- Hoch, J.J., Shamir, A., 2004. Fault analysis of stream ciphers. In: Cryptographic Hardware and Embedded Systems – CHES 2004: 6th International Workshop, vol. 3156 of Lecture Notes in Computer Science. Springer, pp. 240–253.
- Kim, C., Schlaffer, M., Moon, S., 2008. Differential side channel analysis attacks on FPGA implementation of ARIA. ETRI Journal 30 (2), 315–325.
- Kim, C.H., 2010. Differential fault analysis against AES-192 and AES-256 with minimal faults. In: Fault Diagnosis and Tolerance in Cryptography, 7th International Workshop, FDTCT 2010. IEEE Computer Society, pp. 3–9.
- Kim, C.H., Quisquater, J.-J., 2007. Faults, injection methods, and fault attacks. IEEE Design and Test of Computers 24 (6), 544–545.
- Kim, C.H., Quisquater, J.-J., 2008. New differential fault analysis on AES key schedule: Two faults are enough. In: Smart Card Research and Advanced Applications, 8th IFIP WG 8.8/11.2 International Conference, CARDIS 2008, vol. 5189 of Lecture Notes in Computer Science. Springer, pp. 48–60.
- Kocher, P.C., Jaffe, J., Jun, B., 1999. Differential power analysis. In: Advances in Cryptology – CRYPTO '99, 19th Annual International Cryptology Conference, vol. 1666 of Lecture Notes in Computer Science. Springer, pp. 388–397.
- Kwon, D., Kim, J., Park, S., Sung, S.H., Sohn, Y., Song, J.H., Yeom, Y., Yoon, E.-J., Lee, S., Lee, J., Chee, S., Han, D., Hong, J., 2003. New block cipher: ARIA. In: 6th International Conference on Information Security and Cryptology – ICISC 2003, vol. 2971 of Lecture Notes in Computer Science, pages. Springer, pp. 432–445.
- Li, R., Sun, B., Zhang, P., Li, C., 2008. New impossible differential cryptanalysis of ARIA. IACR Eprint Archive 200, 8–227.
- Li, S., Song, C., 2008. Improved impossible differential cryptanalysis of ARIA. In: International Conference on Information Security and Assurance, ISA 2008, pp. 129–132.
- Li, W., Gu, D., Li, J., 2008. Differential fault analysis on the ARIA algorithm. Information Sciences 178 (19), 3727–3737.
- Li, W., Gu, D., Wang, Y., 2009. Differential fault analysis on the contracting UFN structure, with application to SMS4 and Macguffin. Journal of Systems and Software 82 (2), 346–354.
- Moradi, A., Shalmani, M.T.M., Salmasizadeh, M., 2006. A generalized method of differential fault attack against AES cryptosystem. In: Cryptographic Hardware and Embedded Systems – CHES 2006, 8th International Workshop, vol. 4249 of Lecture Notes in Computer Science. Springer, pp. 91–100.
- Naccache, D., Nguyen, P.Q., Tunstall, M., Whelan, C., 2005. Experimenting with faults, lattices and the DSA. In: Public Key Cryptography – PKC 2005, 8th International Workshop on Theory and Practice in Public Key Cryptography, vol. 3386 of Lecture Notes in Computer Science. Springer, pp. 16–28.
- National Institute of Standard and Technology, 1993. Data Encryption Standard, NIST FIPS PUB 46-2.
- Park, J., Lee, H., Ahn, M., 2007. Side-channel attacks against ARIA on active RFID device. In: International Conference on Convergence Information Technology, ICCIT 2007. IEEE Computer Society, pp. 2163–2168.
- Piret, G., Quisquater, J.-J., 2003. A differential fault attack technique against SPN structures, with application to the AES and KHAZAD. In: Cryptographic Hardware and Embedded Systems – CHES 2003, 5th International Workshop, vol. 2779 of Lecture Notes in Computer Science. Springer, pp. 77–88.
- Quisquater, J.-J., Samyde, D., 2001. Electromagnetic analysis (EMA): Measures and counter-measures for smart cards. In: Smart Card Programming and Security, International Conference on Research in Smart Cards, E-smart 2001, vol. 2140 of Lecture Notes in Computer Science. Springer, pp. 200–210.
- Rivain, M., 2009. Differential fault analysis on DES middle rounds. In: Cryptographic Hardware and Embedded Systems – CHES 2009: 11th International Workshop, vol. 5747 of Lecture Notes in Computer Science. Springer, pp. 457–469.
- Saha, D., Mukhopadhyay, D., Roychowdhury, D., 2009. A diagonal fault attack on the advanced encryption standard. IACR Eprint Archive 200, 9–581.
- Schmidt, J.-M., Medwed, M., 2009. A fault attack on ECDSA. In: 6th Fault Diagnosis and Tolerance in Cryptography, Third International Workshop – FDTCT 2009. IEEE Computer Society, pp. 93–99.
- Takahashi, J., Fukunaga, T., 2008. Improved differential fault analysis on CLEFIA. In: Fifth International Workshop on Fault Diagnosis and Tolerance in Cryptography, 2008, FDTCT 2008. IEEE Computer Society, pp. 25–34.
- Takahashi, J., Fukunaga, T., Yamakoshi, K., 2007. DFA mechanism on the AES key schedule. In: 4th International Workshop on Fault Diagnosis and Tolerance in Cryptography, FDTCT 2007. IEEE Computer Society, pp. 62–74.
- Wu, W.-L., Zhang, W.-T., Feng, D.-G., 2007. Impossible differential cryptanalysis of reduced-round ARIA and Camellia. Journal of Computer Science and Technology 22 (3), 449–456.
- Yoo, H., Herbst, C., Mangard, S., Oswald, E., Moon, S.-J., 2006. Investigations of power analysis attacks and countermeasures for ARIA. In: 7th International Workshop on Information Security Applications, WISA 2006, vol. 4298 of Lecture Notes in Computer Science. Springer, pp. 160–172.
- Zhao, X.-J., Wang, T., Gao, J.-Z., 2010. Multiple bytes differential fault analysis on CLEFIA. IACR Eprint Archive, 2010–2078.

Chong Hee Kim received his B.S. from Kyungpook National University, Republic of Korea in 1997, M.S. and Ph.D. from POSTECH (Pohang University of Science and Technology), Republic of Korea in 1999 and 2004, respectively. He worked for Samsung Electronics Co., LTD and NXP Semiconductors N.V. He is currently senior researcher at Universite Catholique de Louvain, Belgium. His recent research interests include RFID secure protocols and security of the embedded systems such as side channel analysis and fault attacks.